

U N D E R S T A N D I N G

DAX

Data Analysis Expressions

A Complete Tutorial: From Fundamentals to Advanced Techniques



01

What is DAX?

Origins, purpose, and where it runs

02

Data Models & Relationships

Tables, columns, keys, and star schemas

03

DAX Syntax Fundamentals

Operators, literals, and expression structure

04

Calculated Columns vs Measures

Row context vs filter context

05

Core Aggregation Functions

SUM, AVERAGE, COUNT, MIN, MAX, DISTINCTCOUNT

06

Filter Functions

CALCULATE, FILTER, ALL, ALLEXCEPT, KEEPFILTERS

07

Iterator Functions (X Functions)

SUMX, AVERAGEX, MAXX, RANKX

08

Time Intelligence

DATEADD, SAMEPERIODLASTYEAR, YTD, MTD

09

Relationship Functions

RELATED, RELATEDTABLE, USERELATIONSHIP

10

Advanced DAX Patterns

Variables, context transition, row-level security

What is DAX?

Origins, purpose, and where DAX runs



What is DAX?: Data Analysis Expressions

4

DAX is a formula language used to define custom calculations in tabular data models.

It powers Power BI, Excel Power Pivot, Analysis Services Tabular, and Azure Analysis Services.



Power BI

Native expression language for all measures and calculated columns in Power BI Desktop & Service



Excel Power Pivot

Extends Excel data models with custom calculations across millions of rows



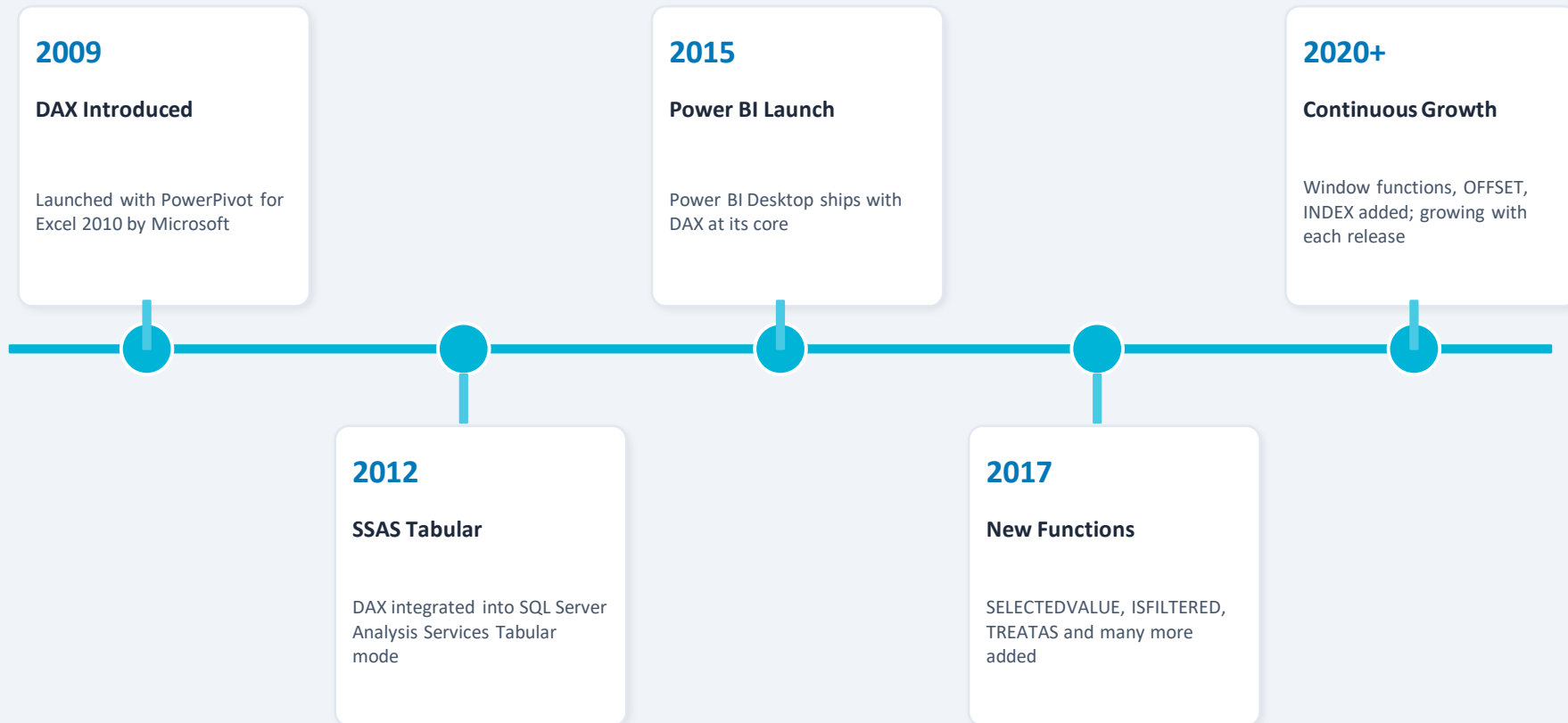
SSAS Tabular

Used in SQL Server Analysis Services (Tabular) and Azure Analysis Services

DAX vs. Excel Formulas: Key Differences

5

Aspect	Excel Formulas	DAX
Operates on	Single cells / ranges	Entire columns & tables
Context awareness	Static cell references	Dynamic filter & row context
Data scale	Thousands of rows	Millions to billions of rows
Relationships	VLOOKUP / INDEX-MATCH	Native relationship navigation
Iteration	Drag-fill / ARRAYFORMULA	Iterator functions (SUMX, etc.)
Where stored	Worksheet cells	Data model measures/columns



The Four Building Blocks of DAX

7



Functions

Pre-built operations like SUM(), CALCULATE(), FILTER(). Over 250 functions available.

e.g. SUM, AVERAGE, IF, CALCULATE



Operators

Mathematical, comparison, and logical operators that combine values and expressions.

e.g. + - * / && || = <>



Values & Literals

Numbers, text strings, dates, and Boolean values used directly in expressions.

e.g. 100, "Sales", TRUE, DATE(2024,1,1)



Column References

Fully qualified table and column references that form the backbone of every DAX expression.

e.g. Sales[Amount], 'Product'[Name]

SECTION 02

Data Models & Relationships

Tables, columns, keys, and star schemas



The Tabular Data Model

9

DAX operates on an in-memory columnar database (VertiPaq engine). Data is organized into tables connected by relationships.

DIMENSION TABLES

Date

DateKey (PK)

Date
Year
Month
Quarter

Product

ProductKey (PK)

ProductName
Category
Price

FACT TABLE

Sales

SalesID (PK)

DateKey (FK)

ProductKey (FK)

CustomerKey (FK)

Amount
Quantity

Customer

CustomerKey (PK)

Name
Country
Segment

★ Star Schema

RECOMMENDED for DAX

- One fact table at center
- Multiple dimension tables
- No joins between dimensions
- Simple, fast filtering
- Optimal VertiPaq compression
- Best DAX performance

❄ Snowflake Schema

AVOID in DAX models

- Dimensions split into sub-dims
- Normalization reduces redundancy
- Increases relationship count
- Harder to write DAX against
- Slower filter propagation
- Flatten before loading into model

DAX SYNTAX – Column & Table References

-- Fully qualified column reference

```
Sales[Amount]
```

-- Table name with spaces must use single quotes

```
'Product Category'[CategoryName]
```

-- In a CALCULATE or table function, reference the table

```
FILTER( Sales, Sales[Amount] > 1000 )
```

-- Inside an iterator, you can omit the table name (current row):

```
SUMX( Sales, [Amount] * [Quantity] )    -- [Amount] = Sales[Amount]
```



Use fully qualified references (Table[Column]) for clarity and to avoid ambiguity errors



Single quotes around table names are required when names contain spaces or special characters



Inside iterator functions (SUMX, FILTER), abbreviated [Column] implicitly refers to the current table

DAX Syntax Fundamentals

Operators, literals, expression structure, and data types



Arithmetic Operators

+	Addition
-	Subtraction
*	Multiplication
/	Division
^	Exponentiation

Logical Operators

&&	AND (short-circuit)
	OR (short-circuit)
!	NOT operator
IN { }	List membership
NOT IN	Not in list

Comparison Operators

=	Equal to
<>	Not equal
>	Greater than
<	Less than
>=	Greater or equal
<=	Less or equal

Text Operators

&	String concatenation
BLANK ()	Empty/null value

Whole Number

64-bit signed integer. No decimals.

```
Sales[Quantity] = 42
```

Currency

Fixed decimal, 4 places. Good for money values.

```
Sales[Revenue] = $1,250.00
```

Boolean

TRUE or FALSE. Result of comparisons.

```
Sales[Amount] > 1000
```

Binary

Binary data. Rare; mainly images via DirectQuery.

```
N/A — system use
```

Decimal Number

64-bit floating-point. Supports fractions.

```
Sales[Price] = 19.99
```

Date/DateTime

Stored internally as a floating-point (OLE date).

```
DATE(2024, 6, 15)
```

Text

Unicode string. Max ~268 MB.

```
"Hello, World!"
```

Blank

DAX null equivalent. BLANK() function or empty.

```
BLANK(), IF(FALSE, 1)
```

COMMENTS & VAR/RETURN PATTERN

```
-- This is a single-line comment

/* This is a
   multi-line comment */

-- Best Practice: Use variables and line breaks for readability
Profit Margin =
VAR TotalRevenue = SUM( Sales[Revenue] )
VAR TotalCost    = SUM( Sales[Cost] )
VAR Profit       = TotalRevenue - TotalCost
RETURN
    DIVIDE( Profit, TotalRevenue, 0 )
```

Formatting Rules

- ✓ Use VAR/RETURN for multi-step calculations
- ✓ One logical step per line
- ✓ Indent continuation lines 4 spaces
- ✓ Comment purpose, not mechanics
- ✓ Use daxformatter.com for auto-formatting
- ✓ Capitalize function names consistently
- ✓ Use blank lines between logical blocks

```
IF / SWITCH / AND / OR - LOGICAL FUNCTIONS
```

```
-- IF( condition, result_if_true, result_if_false )  
Status = IF( Sales[Amount] >= 1000, "High", "Low" )
```

```
-- Nested IF  
Tier = IF( Sales[Amount] >= 5000, "Platinum",  
          IF( Sales[Amount] >= 1000, "Gold", "Standard" ) )
```

```
-- SWITCH - cleaner alternative to nested IF  
Tier = SWITCH( TRUE(),  
              Sales[Amount] >= 5000, "Platinum",  
              Sales[Amount] >= 1000, "Gold",  
              Sales[Amount] >= 500,  "Silver",  
              "Standard"  
            )
```

```
-- AND / OR as functions  
Flag = AND( Sales[Amount] > 500, Sales[Quantity] > 10 )  
Flag = OR( Sales[Region] = "East", Sales[Region] = "West" )
```

Calculated Columns vs Measures



The single most important distinction in DAX

A calculated column is evaluated ROW BY ROW at data refresh time and stored in the model (consumes memory).

CALCULATED COLUMN EXAMPLES

```
-- Add to a table in the model:
-- Example: Profit column on Sales table
Profit = Sales[Revenue] - Sales[Cost]

-- Full Price including Tax
Price with Tax = Sales[UnitPrice] * 1.2

-- Category bucket using IF
Size Category = IF( Products[UnitPrice] > 100, "Premium",
"Standard" )

-- Date part extraction
Year = YEAR( Sales[OrderDate] )
Month Number = MONTH( Sales[OrderDate] )
```

When to use a Calculated Column

- ✓ You need to slice/filter by the new value
- ✓ The calculation needs row-level granularity
- ✓ Used in relationships or hierarchies
- ✓ Category labels, bucketing, date parts
- ✓ No aggregation needed — just per-row values

A measure is evaluated ON DEMAND in a query context , it aggregates values based on whatever filters exist at the time.

MEASURE EXAMPLES

-- Simple aggregation measures:

Total Sales = SUM(Sales[Amount])

Average Price = AVERAGE(Products[Price])

Order Count = COUNTROWS(Sales)

Unique Customers = DISTINCTCOUNT(Sales[CustomerID])

-- Ratio measure:

Profit Margin % =

 DIVIDE(

 SUM(Sales[Profit]),

 SUM(Sales[Revenue]),

 0 -- return 0 if denominator is blank

)

-- Conditional measure:

High Value Orders =

 COUNTROWS(FILTER(Sales, Sales[Amount] > 1000))

When to use a Measure

- ✓ Aggregations: SUM, COUNT, AVERAGE
- ✓ KPIs and ratios shown in visuals
- ✓ Dynamic calculations that change by filter
- ✓ Time intelligence comparisons
- ✓ Values that don't need to be in rows/columns

Calculated Columns vs Measures: Side by Side

20

Attribute	Calculated Column	Measure
Evaluation	Row by row at refresh	On demand per query
Storage	Stored in model (uses RAM)	Not stored — computed live
Context	Row context (current row)	Filter context (visual filters)
Used in	Filters, slicers, rows, columns	Values area of visuals
Aggregation	No — one value per row	Yes — aggregates many rows
Performance	Increases model size	Lightweight (no storage)
When refreshed	At every data refresh	At every query
Best for	Row-level labels & categories	KPIs, ratios, running totals

Understanding Evaluation Contexts

Row Context

Created by calculated columns and iterator functions. DAX knows WHICH ROW is currently being evaluated. Lets you access column values directly: Sales[Amount]

```
Profit Col = Sales[Revenue] - Sales[Cost]
-- [Revenue] and [Cost] refer to CURRENT
ROW
```

Filter Context

The set of filters active at the time a measure is evaluated. Comes from slicers, visual filters, rows/columns in a matrix, and CALCULATE() function.

```
Total Sales = SUM( Sales[Amount] )
-- "Sales" is filtered by Year=2024,
Region="East"
```

Core Aggregation Functions

SUM, AVERAGE, COUNT, MIN, MAX, DISTINCTCOUNT and more



SUM vs SUMX

-- SUM: adds up all values in a single column

Total Revenue = SUM(Sales[Revenue])

-- SUMX: iterates rows, evaluates expression, then sums

-- Use when the value doesn't exist as a column

Total Revenue =

SUMX(

Sales, -- table to iterate

Sales[Quantity] * Sales[Price] -- expression per row

)

-- When are they different?

-- If Revenue column exists: SUM(Sales[Revenue]) is identical

-- to SUMX(Sales, Sales[Revenue])

-- But SUMX can handle row-level calculations:

Discounted Rev =

SUMX(Sales, Sales[Price] * Sales[Qty] * (1 - Sales[Disc]))

COUNT (col)

Counts numeric values only. Ignores text, blanks.

```
COUNT( Sales[Amount] )
```

COUNTA (col)

Counts non-blank values in any data type (text, number, date).

```
COUNTA( Customers[Name] )
```

COUNTROWS (tbl)

Counts the total number of rows in a table or table expression.

```
COUNTROWS( Sales )
```

COUNTBLANK (col)

Counts blank (null) values in a column.

```
COUNTBLANK( Sales[Region] )
```

DISTINCTCOUNT (col)

Counts unique non-blank values — equivalent to COUNT DISTINCT.

```
DISTINCTCOUNT( Sales[CustomerID] )
```

DISTINCTCOUNTNOBLANK (col)

Like DISTINCTCOUNT but strictly excludes blanks.

```
DISTINCTCOUNTNOBLANK( Sales[ID] )
```

AVERAGE / MIN / MAX VARIANTS

-- AVERAGE / AVERAGEX

Avg Order Value = AVERAGE(Sales[Amount])

Avg Weighted = AVERAGEX(Sales, Sales[Price] * Sales[Qty])

-- MIN / MAX – simple column min/max

First Order Date = MIN(Sales[OrderDate])

Highest Sale = MAX(Sales[Amount])

-- MINX / MAXX – min/max of an expression across rows

Cheapest Bundle =

MINX(Products, Products[Price] * Products[MinQty])

-- MIN / MAX between two scalars (not columns!)

Capped Amount =

SUMX(Sales, MIN(Sales[Amount], 500)) -- cap each row at 500

-- MEDIAN (no MEDIANX in older versions – use PERCENTILE)

Median Sales = MEDIAN(Sales[Amount])

P90 Sales = PERCENTILE.INC(Sales[Amount], 0.9)

MATH FUNCTIONS

```
-- DIVIDE( numerator, denominator [, alternate] )
-- Safe division - returns alternate if denominator = 0
Margin % = DIVIDE( SUM(Sales[Profit]), SUM(Sales[Revenue]), 0 )

-- ROUND, ROUNDUP, ROUNDDOWN, MROUND
Rounded = ROUND( [Avg Price], 2 )           -- round to 2 decimals
Ceiling = ROUNDUP( [Avg Price], 0 )         -- always round up
Floor   = ROUNDDOWN( [Avg Price], 0 )       -- always round down
Near5   = MROUND( [Amount], 5 )            -- round to nearest 5

-- ABS, INT, MOD
Variance  = ABS( [Actual] - [Budget] )
Whole Part = INT( Sales[Price] )
Remainder = MOD( Sales[Rank], 3 )          -- for alternating groups

-- POWER, SQRT, EXP, LOG
Squared = POWER( Sales[Value], 2 )
SqRoot  = SQRT( Sales[Area] )
```

Filter Functions

CALCULATE, FILTER, ALL, ALLEXCEPT — the heart of DAX



CALCULATE: The Most Important DAX Function

```
CALCULATE( <expression>, <filter1>, <filter2>, ... )
```

CALCULATE SYNTAX & EXAMPLES

```
-- CALCULATE evaluates expression in a MODIFIED filter context

-- Example 1: Filter by a specific value
East Sales = CALCULATE( SUM(Sales[Amount]), Sales[Region] = "East" )

-- Example 2: Multiple filters (both must be true = AND)
East 2024 = CALCULATE(
    SUM( Sales[Amount] ),
    Sales[Region] = "East",
    'Date'[Year] = 2024
)

-- Example 3: Override existing filter with ALL
Total All Regions = CALCULATE( SUM(Sales[Amount]), ALL(Sales[Region]) )

-- Example 4: Using a FILTER expression
Big Orders = CALCULATE(
    SUM( Sales[Amount] ),
    FILTER( Sales, Sales[Amount] > 1000 )
)
```

FILTER FUNCTION

```
-- FILTER returns a TABLE (subset of rows matching condition)
-- Syntax: FILTER( <table>, <condition> )

-- Used as argument inside CALCULATE or iterators:
Premium Products =
    FILTER( Products, Products[Price] > 100 )

-- FILTER inside CALCULATE:
Premium Revenue =
    CALCULATE (
        SUM( Sales[Amount] ),
        FILTER( Products, Products[Price] > 100 )
    )

-- Multi-condition FILTER:
High Value New =
    FILTER(
        Customers,
        Customers[TotalSpend] > 5000 &&
        Customers[JoinYear] >= 2023
    )

-- ⚠️ Prefer CALCULATE( expr, Table[Col] = value ) over FILTER for simple cases
-- FILTER iterates all rows – slower on large tables
```

```
ALL / ALLEXCEPT / ALLSELECTED
```

```
-- ALL( table_or_column ) - removes filters from table or column
```

```
% of Total Sales =
```

```
    DIVIDE(
        SUM( Sales[Amount] ),
        CALCULATE( SUM(Sales[Amount]), ALL(Sales) ),
        0
    )
```

```
-- ALLEXCEPT - remove all filters EXCEPT specified columns
```

```
% of Region Total =
```

```
    DIVIDE(
        SUM( Sales[Amount] ),
        CALCULATE( SUM(Sales[Amount]), ALLEXCEPT(Sales, Sales[Region]) ),
        0
    )
```

```
-- ALLSELECTED - respects visual-level filters only (ignores cross-filters)
```

```
% of Visual Total =
```

```
    DIVIDE(
        SUM( Sales[Amount] ),
        CALCULATE( SUM(Sales[Amount]), ALLSELECTED() ),
        0
    )
```

```
KEEPFILTERS / REMOVEFILTERS / CROSSFILTER
```

```
-- KEEPFILTERS: Intersects new filter with existing filter context  
-- Instead of replacing, it adds the constraint
```

```
East Sales (safe) =  
    CALCULATE(  
        SUM( Sales[Amount] ),  
        KEEPFILTERS( Sales[Region] = "East" )  
    )
```

```
-- REMOVEFILTERS( col ) - same as ALL on a specific column
```

```
Total Ignoring Product =  
    CALCULATE( SUM(Sales[Amount]), REMOVEFILTERS(Products) )
```

```
-- CROSSFILTER - controls relationship direction inside CALCULATE
```

```
-- CROSSFILTER( col1, col2, direction )
```

```
-- direction: NONE | BOTH | ONEWAY
```

```
Customers With Orders =  
    CALCULATE(  
        COUNTROWS( Customers ),  
        CROSSFILTER( Sales[CustomerID], Customers[CustomerID], BOTH )  
    )
```

```
VALUES / DISTINCT / HASONEVALUE / SELECTEDVALUE

-- VALUES( column ) - unique values in current filter context (includes blank)
Years Available = COUNTROWS( VALUES( 'Date'[Year] ) )

-- DISTINCT( column ) - unique values (excludes blank from relationships)
Product Count = COUNTROWS( DISTINCT( Products[ProductID] ) )

-- HASONEVALUE( column ) - TRUE if exactly one value in context
Label =
    IF(
        HASONEVALUE( 'Date'[Year] ),
        "Year: " & VALUES( 'Date'[Year] ),
        "Multiple Years"
    )

-- SELECTEDVALUE( column [, else] ) - cleaner pattern
Selected Year = SELECTEDVALUE( 'Date'[Year], "All Years" )

-- ISINSCOPE - check if a column is in the current grouping
Label =
    IF( ISINSCOPE('Date'[Month]), "Month",
        IF( ISINSCOPE('Date'[Year]), "Year", "Total" ) )
```

Iterator (X) Functions

Row-by-row evaluation with SUMX, AVERAGEX, RANKX and more



Iterator functions loop over a table row by row, evaluate an expression in each row's ROW CONTEXT, then aggregate the results.

Pattern: `XFUNCTION( <table>, <expression> )`

- 1 Take the table (or filtered subset)
- 2 Loop: go to row 1, evaluate <expression>
- 3 Move to row 2, evaluate <expression>
- 4 Repeat for all rows ...
- 5 Aggregate all results (sum / avg / max / etc.)

SUMX PATTERNS

```
-- Simple SUMX (equivalent to SUM when col exists)
Total Qty = SUMX( Sales, Sales[Quantity] )

-- SUMX with expression (no pre-calculated column needed)
Total Revenue =
    SUMX( Sales, Sales[UnitPrice] * Sales[Quantity] )

-- SUMX with discount
Net Revenue =
    SUMX( Sales,
        Sales[UnitPrice] * Sales[Quantity] * (1 - Sales[Discount]) )

-- SUMX iterating a FILTER result
Premium Rev =
    SUMX(
        FILTER( Sales, Sales[Category] = "Premium" ),
        Sales[UnitPrice] * Sales[Quantity]
    )

-- SUMX with RELATED (fetching from related table)
Cost of Sales =
    SUMX( Sales, Sales[Quantity] * RELATED( Products[StandardCost] ) )
```

RANKX — RANKING FUNCTION

```
-- RANKX( <table>, <expression>, [<value>], [<order>], [<ties>] )

-- Rank products by total sales (1 = highest)
Sales Rank =
  RANKX(
    ALL( Products ),           -- rank across ALL products
    [Total Sales],             -- the measure to rank by
    ,                           -- omit value (use current context)
    DESC,                       -- descending order
    DENSE                       -- DENSE = no gaps in ranks
  )

-- DENSE vs SKIP for ties:
--   DENSE:  1, 2, 2, 3, 4   (no gaps)
--   SKIP:   1, 2, 2, 4, 5   (skips 3 after a tie)

-- Rank within group (e.g. rank within category)
Rank in Category =
  RANKX(
    ALLEXCEPT( Products, Products[Category] ),
    [Total Sales]
  )
```

TOPN / MAXX / MINX / COUNTX

-- TOPN(n, table, orderBy [, order]) – returns top N rows

Top 5 Products =

TOPN(5, Products, [Total Sales], DESC)

-- Useful in CALCULATE:

Top 5 Revenue =

CALCULATE(

SUM(Sales[Amount]),

TOPN(5, Products, [Total Sales], DESC)

)

-- MAXX / MINX – max/min of an expression across rows

Highest Order Value =

MAXX(Sales, Sales[UnitPrice] * Sales[Quantity])

-- COUNTX – count rows where expression is non-blank

Orders Over 500 = COUNTX(FILTER(Sales, Sales[Amount]>500), Sales[SalesID])

-- PRODUCTX – multiply all row values together

Compound Growth =

PRODUCTX(GrowthTable, 1 + GrowthTable[Rate])

SECTION 08

Time Intelligence

Date calculations: YTD, MTD, SAMEPERIODLASTYEAR, and more





Time Intelligence requires a proper Date table, without it, functions like SAMEPERIODLASTYEAR will return incorrect results!

1 Contiguous dates: Must have EVERY date from start to end of range — no gaps allowed

2 Marked as Date Table: Right-click table → Mark as Date Table (in Power BI or SSAS)

3 Date column: Must have a Date (not DateTime) column designated as the key

4 One row per date: Each calendar date appears exactly once — no duplicates

5 Relationship to facts: Many-to-one relationship from fact table's date column to Date[Date]

```
YTD / QTD / MTD FUNCTIONS

-- TOTALYTD / TOTALQTD / TOTALMTD
Sales YTD =
    TOTALYTD( SUM(Sales[Amount]), 'Date'[Date] )

Sales QTD =
    TOTALQTD( SUM(Sales[Amount]), 'Date'[Date] )

Sales MTD =
    TOTALMTD( SUM(Sales[Amount]), 'Date'[Date] )

-- Fiscal Year YTD (FY ending June 30)
Fiscal YTD =
    TOTALYTD( SUM(Sales[Amount]), 'Date'[Date], "06/30" )

-- Manual approach (more flexible, same result):
Sales YTD v2 =
    CALCULATE(
        SUM( Sales[Amount] ),
        DATESYTD( 'Date'[Date] )
    )
```

```
SAMEPERIODELASTYEAR / DATEADD

-- SAMEPERIODELASTYEAR - returns same dates, one year back
Sales LY =
    CALCULATE(
        SUM( Sales[Amount] ),
        SAMEPERIODELASTYEAR( 'Date'[Date] )
    )

-- YoY Change Amount
YoY Change =
    SUM( Sales[Amount] ) - [Sales LY]

-- YoY % Change
YoY % Change =
    DIVIDE( [YoY Change], [Sales LY], BLANK() )

-- PREVIOUSYEAR - entire previous calendar year
Sales Prev Year =
    CALCULATE(
        SUM( Sales[Amount] ),
        PREVIOUSYEAR( 'Date'[Date] )
    )

-- DATEADD - shift by any number of intervals
Sales 3M Ago =
    CALCULATE( SUM(Sales[Amount]), DATEADD('Date'[Date], -3, MONTH) )
```

DATESINPERIOD / DATESBETWEEN

```
-- DATESINPERIOD( dates, start, intervals, interval_type )
-- Returns a set of dates in a period

-- Rolling 30-day sales:
Sales Rolling 30D =
    CALCULATE(
        SUM( Sales[Amount] ),
        DATESINPERIOD( 'Date'[Date], MAX('Date'[Date]), -30, DAY )
    )

-- Rolling 3-month sales:
Sales Rolling 3M =
    CALCULATE(
        SUM( Sales[Amount] ),
        DATESINPERIOD( 'Date'[Date], MAX('Date'[Date]), -3, MONTH )
    )

-- DATESBETWEEN( dates, start, end )
Holiday Sales =
    CALCULATE(
        SUM( Sales[Amount] ),
        DATESBETWEEN( 'Date'[Date],
            DATE(2024, 11, 29),    -- Black Friday
            DATE(2024, 12, 31)    -- Year end
        )
    )
```

```
PARALLELPERIOD / FIRSTDATE / LASTDATE / LASTNONBLANK
```

```
-- PARALLELPERIOD( dates, offset, interval ) - entire parallel period  
-- Useful for full prior period (vs partial with DATEADD)
```

```
Sales Prior Quarter =
```

```
    CALCULATE(  
        SUM( Sales[Amount] ),  
        PARALLELPERIOD( 'Date'[Date], -1, QUARTER )  
    )
```

```
-- FIRSTDATE / LASTDATE - first/last date in current filter context
```

```
Opening Balance =
```

```
    CALCULATE(  
        SUM( Balance[Amount] ),  
        FIRSTDATE( 'Date'[Date] )  
    )
```

```
Closing Balance =
```

```
    CALCULATE(  
        SUM( Balance[Amount] ),  
        LASTDATE( 'Date'[Date] )  
    )
```

```
-- LASTNONBLANK - last date that has data
```

```
Last Sale Date =
```

```
    LASTNONBLANK( 'Date'[Date], SUM(Sales[Amount]) )
```

Relationship Functions

RELATED, RELATEDTABLE, USERRELATIONSHIP, TREATAS



RELATED - LOOKUP FROM MANY-TO-ONE

```
-- RELATED( column ) - fetches ONE value from the 'one' side
-- Requires a many-to-one relationship (current table is 'many')

-- Example: Sales table needs Product Category
-- Relationship: Sales[ProductKey] → Products[ProductKey]

-- In a Calculated Column on Sales table:
Product Category = RELATED( Products[Category] )

-- In SUMX to access related column in expression:
Revenue Above Cost =
    SUMX(
        Sales,
        IF(
            Sales[UnitPrice] > RELATED( Products[StandardCost] ),
            Sales[UnitPrice] * Sales[Quantity],
            0
        )
    )

-- Can chain multiple hops (if relationships exist):
Sub Category = RELATED( Products[SubCategory] ) -- Products → SubCategory
```

RELATEDTABLE – AGGREGATE FROM ONE-TO-MANY

```
-- RELATEDTABLE( table ) – returns all related rows from 'many' side
-- Used from the 'one' side of a relationship
```

```
-- Example: Count orders per customer (on Customers table)
Order Count = COUNTROWS( RELATEDTABLE( Sales ) )
```

```
-- Total revenue per product (on Products table)
Product Revenue =
    SUMX(
        RELATEDTABLE( Sales ),
        Sales[UnitPrice] * Sales[Quantity]
    )
```

```
-- Check if customer has any premium order
Has Premium Order =
    COUNTROWS(
        FILTER(
            RELATEDTABLE( Sales ),
            Sales[Amount] > 1000
        )
    ) > 0
```

```
-- RELATEDTABLE vs CALCULATETABLE:
-- They're identical in a calculated column context
-- CALCULATETABLE is more flexible in measures
```

```
USERELATIONSHIP / TREATAS
```

```
-- Scenario: Sales table has two date columns:
```

```
--   Sales[OrderDate]   → 'Date'[Date]   (ACTIVE)
```

```
--   Sales[ShipDate]    → 'Date'[Date]   (INACTIVE – only one can be active)
```

```
-- Default measure uses OrderDate (active relationship):
```

```
Sales by Order Date = SUM( Sales[Amount] )
```

```
-- Activate ShipDate relationship inside CALCULATE:
```

```
Sales by Ship Date =
```

```
    CALCULATE (
        SUM( Sales[Amount] ),
        USERELATIONSHIP( Sales[ShipDate], 'Date'[Date] )
    )
```

```
-- TREATAS – treat a column as if it were related
```

```
-- Useful when no physical relationship exists
```

```
Budget for Actuals =
```

```
    CALCULATE (
        SUM( Budget[Amount] ),
        TREATAS( VALUES('Date'[Year]), Budget[Year] )
    )
```

SECTION 10

Advanced DAX Patterns

Variables, context transition, RLS, and expert-level patterns



VAR / RETURN: Variables in DAX

Variables capture intermediate results, improving readability and performance. Once assigned, a VAR is calculated ONCE and reused.

VAR / RETURN PATTERN

```
-- Without VAR: repeated calculations
Bad Margin =
    DIVIDE( SUM(Sales[Revenue]) - SUM(Sales[Cost]),
            SUM(Sales[Revenue]), 0 )

-- With VAR: clear, calculated once
Profit Margin =
VAR Rev      = SUM( Sales[Revenue] )
VAR Cost     = SUM( Sales[Cost] )
VAR Profit   = Rev - Cost
RETURN
    DIVIDE( Profit, Rev, 0 )

-- VAR for conditional branching (avoids double evaluation):
Status =
VAR Score = [Total Score]
RETURN
    SWITCH( TRUE(),
        Score >= 90, "Excellent",
        Score >= 70, "Good",
        Score >= 50, "Average",
        "Below Average"
```

Context Transition: The Hidden Power of CALCULATE

When CALCULATE is called inside a row context (e.g. inside SUMX), it CONVERTS the row context into a filter context — this is Context Transition.

CONTEXT TRANSITION

```
-- Scenario: Calculated column on Customers table
-- [Total Sales] is a MEASURE

-- This WORKS — CALCULATE triggers context transition:
-- Row context (CustomerID=42) → filter context { CustomerID=42 }
Customer Sales = CALCULATE( [Total Sales] )

-- Inside SUMX, calling a measure also triggers transition:
Weighted Score =
    SUMX(
        Customers,
        Customers[Weight] * CALCULATE( [Total Sales] )
        -- CALCULATE converts current Customers row to filter
    )

-- ⚠ Without CALCULATE, measure has no row filter:
Wrong =
    SUMX( Customers, Customers[Weight] * [Total Sales] )
    -- [Total Sales] uses OUTER filter context, not each customer!
```

EARLIER / NESTED ROW CONTEXTS

```
-- EARLIER( column ) - accesses the OUTER row context
-- Used in calculated columns to compare a row with others

-- Count how many products cost less than the current product:
Cheaper Count =
    COUNTROWS (
        FILTER (
            Products,
            Products[Price] < EARLIER( Products[Price] )
        )
    )

-- Running rank using EARLIER:
Price Rank =
    COUNTROWS (
        FILTER (
            Products,
            Products[Price] >= EARLIER( Products[Price] )
        )
    )

-- i In modern DAX, VAR is often cleaner than EARLIER:
Price Rank V2 =
    VAR CurrentPrice = Products[Price]
    RETURN COUNTROWS ( FILTER ( Products, Products[Price] >= CurrentPrice ) )
```

RLS restricts data visible to specific users by applying a DAX filter to a table. Defined in Power BI Desktop or SSAS.

ROW-LEVEL SECURITY DAX PATTERNS

```
-- Static RLS: hardcoded filter on Employees role
[Region] = "East"

-- Dynamic RLS: filter based on logged-in user
-- On Sales table, filter by matching email from security table:
[ManagerEmail] = USERPRINCIPALNAME()

-- More complex: user sees their region only
-- Add filter expression to the Sales table in the role:
Sales[Region] =
    LOOKUPVALUE (
        UserRegions[Region],
        UserRegions[UserEmail], USERPRINCIPALNAME()
    )

-- Hierarchy RLS: user sees their team + sub-teams
-- Use PATH() and PATHCONTAINS() for org hierarchy:
PATHCONTAINS (
    LOOKUPVALUE ( Employees[Path], Employees[Email], USERPRINCIPALNAME() ),
    Employees[EmployeeID]
```

TABLE CONSTRUCTOR FUNCTIONS

-- CALENDAR(start_date, end_date) - generate a date table

DateTable =

VAR StartDate = DATE(2020, 1, 1)

VAR EndDate = TODAY()

RETURN CALENDAR(StartDate, EndDate)

-- CALENDARAUTO - automatically determines start/end from model

AutoDate = CALENDARAUTO()

-- GENERATESERIES(start, end, step) - numeric sequence

Numbers = GENERATESERIES(1, 100, 1) -- 1 to 100

Buckets = GENERATESERIES(0, 1000, 50) -- 0, 50, 100, ..., 1000

-- ROW - creates a single-row table

Summary =

ROW(

"Total", SUM(Sales[Amount]),

"Count", COUNTROWS(Sales)

)

-- DATATABLE - hardcoded lookup table

Tiers = DATATABLE(

"Min", INTEGER, "Max", INTEGER, "Label", STRING,

{ { 0, 499, "Bronze" }, { 500, 1999, "Silver" }, { 2000, 9999, "Gold" } }

)

ADVANCED CALCULATE FILTER ARGUMENTS

```
-- Predicate filter (simpler than FILTER for column = value):
East Sales = CALCULATE( [Total Sales], Sales[Region] = "East" )

-- IN operator - filter by a list of values
Key Region Sales =
    CALCULATE(
        [Total Sales],
        Sales[Region] IN { "East", "West", "North" }
    )

-- NOT IN:
Excl Online Sales =
    CALCULATE(
        [Total Sales],
        NOT Sales[Channel] IN { "Online", "App" }
    )

-- Multiple filters (AND between them):
Q1 East 2024 =
    CALCULATE(
        [Total Sales],
        'Date'[Year] = 2024,
        'Date'[Quarter] = 1,
        Sales[Region] = "East"
    )
```

Window Functions: WINDOW, INDEX, OFFSET (Modern DAX)

55

```
WINDOW / OFFSET / INDEX - MODERN DAX
```

```
-- Modern DAX (Power BI 2022+) added SQL-like window functions
```

```
-- OFFSET - access a row offset from current (like LAG/LEAD in SQL)
```

```
Sales Prev Month =
```

```
    CALCULATE(
        [Total Sales],
        OFFSET(-1, ALLSELECTED('Date'), ORDERBY('Date' [MonthNum]))
    )
```

```
-- INDEX - access specific rank position
```

```
Top Product Sales =
```

```
    CALCULATE(
        [Total Sales],
        INDEX(1, ALLSELECTED(Products), ORDERBY([Total Sales], DESC))
    )
```

```
-- WINDOW - returns a table based on relative position range
```

```
Rolling 3 Month =
```

```
    CALCULATE(
        [Total Sales],
        WINDOW(-2, REL, 0, REL,
            ALLSELECTED('Date'),
            ORDERBY('Date' [MonthNum], ASC))
    )
```

TEXT FUNCTIONS

```
-- CONCATENATE / & operator
Full Name = Customers[FirstName] & " " & Customers[LastName]
Full Name = CONCATENATE( Customers[FirstName], " " )

-- UPPER, LOWER, LEN, TRIM, SUBSTITUTE
Upper Name  = UPPER( Customers[Name] )
Clean Email = LOWER( TRIM( Customers[Email] ) )
Name Length = LEN( Customers[Name] )
Fixed Phone = SUBSTITUTE( Customers[Phone], "-", "" )

-- LEFT, RIGHT, MID
Country Code = LEFT( Products[SKU], 2 )
Last4 = RIGHT( Orders[Ref], 4 )
Segment = MID( Products[Code], 3, 5 )

-- FIND, SEARCH, CONTAINS
Position = FIND( "@", Customers[Email] )
HasEmail = ISNUMBER( FIND( "@", Customers[Email] ) )

-- FORMAT - convert numbers/dates to text
Date Label  = FORMAT( 'Date'[Date], "MMM YYYY" )
Amount Text = FORMAT( Sales[Amount], "$#,##0.00" )
```

LOOKUPVALUE FUNCTION

```
-- LOOKUPVALUE( result_col, search_col, search_val )
-- Works WITHOUT a relationship (unlike RELATED)

-- Get product price from Products table:
Item Price =
    LOOKUPVALUE(
        Products[Price],           -- column to return
        Products[ProductCode],    -- column to search in
        Sales[ProductCode]        -- value to look for
    )

-- Multiple search criteria:
Regional Budget =
    LOOKUPVALUE(
        Budget[Amount],
        Budget[Region], Sales[Region],
        Budget[Year],   YEAR( Sales[OrderDate] )
    )

-- Alternative: CALCULATE with filter (often faster)
Item Price V2 =
    CALCULATE(
        MIN( Products[Price] ),
        FILTER( Products, Products[Code] = Sales[Code] )
    )
```

INFORMATION FUNCTIONS

```
-- ISBLANK( value ) - TRUE if value is BLANK()
Has Region = NOT ISBLANK( Sales[Region] )

-- ISNUMBER, ISTEXT, ISLOGICAL
Is Valid Amount = ISNUMBER( Sales[Amount] )

-- ISERROR - catches division errors, type mismatches
Safe Calc =
    IF(
        ISERROR( Sales[Revenue] / Sales[Units] ),
        0,
        Sales[Revenue] / Sales[Units]
    )

-- IFERROR - shorthand: value, else_on_error
Safe Ratio = IFERROR( [Revenue] / [Units], 0 )

-- HASONEVALUE, HASONEFILTER
Chart Title =
    IF(
        HASONEVALUE( 'Date'[Year] ),
        "Data for " & SELECTEDVALUE( 'Date'[Year] ),
        "All Years"
    )
```

CALCULATETABLE

```
-- CALCULATETABLE( <table>, <filter1>, ... )
-- Returns a filtered TABLE (vs CALCULATE which returns a scalar)

-- Get premium products table:
Premium Products =
    CALCULATETABLE(
        Products,
        Products[Price] > 100,
        Products[Category] = "Electronics"
    )

-- Use in SUMX:
Premium Revenue =
    SUMX(
        CALCULATETABLE(
            Sales,
            Sales[Amount] > 500
        ),
        Sales[Amount]
    )

-- i CALCULATETABLE vs FILTER:
-- CALCULATETABLE also supports ALL, ALLEXCEPT, USERELATIONSHIP
-- FILTER is a plain row iterator (no filter modifier support)
-- CALCULATETABLE is generally faster for complex scenarios
```

SEMI-ADDITIVE MEASURE PATTERNS

```
-- Semi-additive: sum across products but NOT across time
-- Correct for inventory or bank balances
```

```
-- LASTNONBLANK pattern (most common):
```

```
Inventory Balance =
    CALCULATE (
        SUM( Inventory[Qty] ),
        LASTNONBLANK (
            'Date'[Date],
            CALCULATE ( SUM( Inventory[Qty] ) )
        )
    )
```

```
-- Opening balance at start of period:
```

```
Opening Stock =
    CALCULATE (
        SUM( Inventory[Qty] ),
        FIRSTNONBLANK (
            'Date'[Date],
            CALCULATE ( SUM( Inventory[Qty] ) )
        )
    )
```

```
-- Headcount at end of month:
```

```
Headcount EOM =
    CALCULATE (
        DISTINCTCOUNT( Employees[EmployeeID] ),
        LASTDATE( 'Date'[Date] )
    )
```

RUNNING TOTAL PATTERN

-- Running Total (Cumulative Sum) pattern:

```
Sales Running Total =  
    CALCULATE(  
        SUM( Sales[Amount] ),  
        FILTER(  
            ALL( 'Date' ),  
            'Date'[Date] <= MAX( 'Date'[Date] )  
        )  
    )
```

-- Using DATESYTD for YTD running total (resets each year):

```
YTD Sales = CALCULATE( SUM(Sales[Amount]), DATESYTD('Date'[Date]) )
```

-- Running count of customers:

```
Cumulative Customers =  
    CALCULATE(  
        DISTINCTCOUNT( Sales[CustomerID] ),  
        FILTER(  
            ALL( 'Date' ),  
            'Date'[Date] <= MAX( 'Date'[Date] )  
        )  
    )
```

-- Performance tip: use ALLSELECTED instead of ALL

-- to respect visual slicer ranges while still summing forward

3-MONTH MOVING AVERAGE

```
-- 3-Month Moving Average:
MA 3M Sales =
VAR CurrentDate = MAX( 'Date' [Date] )
VAR Last3Months =
    DATESINPERIOD (
        'Date' [Date],
        CurrentDate,
        -3,
        MONTH
    )
VAR MonthsWithData =
    CALCULATE (
        DISTINCTCOUNT( 'Date' [YearMonth] ),
        Last3Months
    )
RETURN
    DIVIDE (
        CALCULATE( SUM( Sales[Amount] ), Last3Months ),
        MonthsWithData,
        BLANK()
    )
```

Moving Average Tips



DATESINPERIOD handles month boundaries correctly



Divide by actual months with data — not always 3



Use BLANK() (not 0) when insufficient data exists



Extend to 6M or 12M by changing the -3 parameter



Apply to any measure (revenue, cost, headcount)



Consider using OFFSET for cleaner syntax in newer versions

```
PATH / PATHITEM / PATHCONTAINS - HIERARCHY FUNCTIONS

-- For self-referencing tables (Employees, Accounts, Org Charts)
-- Each row has an ID and a ParentID

-- PATH( id_column, parent_id_column )
-- Returns a delimited string showing the hierarchy path
Employee Path = PATH( Employees[EmployeeID], Employees[ManagerID] )
-- Example result: "1|5|12|47" (root|...|self)

-- PATHLENGTH - how many levels deep
Org Level = PATHLENGTH( Employees[Path] )

-- PATHITEM - extract a specific level
Level1 Manager = PATHITEM( Employees[Path], 1, INTEGER )
Level2 Manager = PATHITEM( Employees[Path], 2, INTEGER )

-- PATHCONTAINS - is a specific ID in the path?
Reports to CEO =
    PATHCONTAINS( Employees[Path], 1 ) -- 1 = CEO's EmployeeID

-- PATHLOOKUP variant using PATHITEMREVERSE:
Direct Manager = PATHITEMREVERSE( Employees[Path], 2, INTEGER )
```

USERPRINCIPALNAME, USERNAME: Dynamic User Context

USER CONTEXT FUNCTIONS

```
-- USERPRINCIPALNAME() - returns logged-in user's email (UPN)
-- Available in Power BI Service; returns empty in Desktop
Current User = USERPRINCIPALNAME()    -- e.g. "alice@company.com"

-- USERNAME() - returns domain\username in SSAS/Power BI RS
Current User SSAS = USERNAME()        -- e.g. "DOMAIN\alice"

-- Dynamic RLS filter on Sales table:
'Date'[Year] >=
    LOOKUPVALUE(
        UserAccess[MinYear],
        UserAccess[Email], USERPRINCIPALNAME()
    )

-- Personalized greeting measure:
Welcome Message =
VAR User = USERPRINCIPALNAME()
VAR Name = LOOKUPVALUE( Employees[FirstName], Employees[Email], User )
RETURN
    IF(
        ISBLANK( Name ),
        "Welcome!",
        "Welcome back, " & Name & "!"
    )
```

DISCONNECTED TABLES / WHAT-IF PARAMETERS

```
-- Disconnected table: a table with NO relationship to the model
-- Used for user-controlled parameters, scenario analysis

-- Example: Discount Rate slicer table (created manually):
-- Table: DiscountRate = GENERATESERIES( 0, 0.5, 0.05 )
-- Column: [Rate] with values 0%, 5%, 10%, ..., 50%

-- Measure uses SELECTEDVALUE to pick user's selection:
Selected Discount =
    SELECTEDVALUE( DiscountRate[Rate], 0.1 ) -- default 10%

-- Apply to sales calculation:
Discounted Revenue =
VAR Disc = [Selected Discount]
RETURN
    SUMX( Sales, Sales[Amount] * (1 - Disc) )

-- Power BI built-in What-If Parameter does this automatically:
-- Modeling → New Parameter → creates the table + measure for you
```



Use a dedicated measure table

Create an empty table (no data) named '_Measures'. Store all measures there. Keeps model tidy.



Prefix measures by category

Use prefixes: "# Orders", "\$ Revenue", "% Margin". Makes slicers and the field list self-documenting.



Base measures first

Write atomic base measures (Total Sales = SUM...) then build complex measures on top. Easier to debug.



Format all measures

Set format strings. \$#,##0 for currency, 0.0% for percentages. Prevents inconsistent formatting.



Use DIVIDE not (/)

Always use DIVIDE() to handle division by zero. DIVIDE returns BLANK() or a specified alternate, / throws an error.



Comment complex measures

Add -- comments explaining why, not just what. Future you will be grateful.

DAX Performance Optimization

Avoid row-by-row in columns

Replace complex calculated columns with simpler ones or move logic to Power Query (M language). Less work at refresh.

Use SUM over SUMX when possible

SUM(col) is a native aggregation. SUMX iterates all rows only use when the expression can't be pre-calculated.

Avoid FILTER in CALCULATE

Prefer predicate syntax: CALCULATE([x], Sales[Region]="East") over FILTER(Sales, ...). Predicates push to VertiPaq engine.

Use variables (VAR)

Variables are calculated once, not re-evaluated. Writing a complex expression twice doubles the computation.

Avoid bidirectional relationships

Two-way filters slow down queries. Use CROSSFILTER() inside CALCULATE only when needed.

Use DAX Studio to profile

DAX Studio (free tool) shows query plans, server timings, and storage engine hits — essential for finding bottlenecks.

ERROR HANDLING & BLANK MANAGEMENT

```
-- IFERROR( expression, value_if_error )
Safe Ratio = IFERROR( [Sales] / [Target], 0 )

-- ISERROR( expression ) - returns TRUE if expression errors
Has Error = ISERROR( VALUE( SomeTable[TextCol] ) )

-- DIVIDE - recommended over / operator
Margin = DIVIDE( [Profit], [Revenue], BLANK() )
--          DIVIDE( numerator, denominator, alternate_result )

-- BLANK() propagation:
-- BLANK + number = number (blank acts as 0 in addition)
-- BLANK * number = BLANK (blank propagates in multiplication)
-- BLANK / anything = BLANK

-- Handling BLANK explicitly:
Sales or Zero = IF( ISBLANK( [Total Sales] ), 0, [Total Sales] )
-- Shorthand:
Sales or Zero = [Total Sales] + 0    -- BLANK + 0 = 0

-- IF with BLANK return (hides zeros in charts):
Sales No Zero = IF( [Total Sales] = 0, BLANK(), [Total Sales] )
```

SWITCH TRUE(): The Ultimate Conditional Pattern

69

```
SWITCH / SWITCH(TRUE()) PATTERN

-- SWITCH( <expression>, <value1>, <result1>, ... [, <else>] )

-- Basic SWITCH — match a value:
Region Code =
    SWITCH(
        Sales[RegionID],
        1, "North",
        2, "South",
        3, "East",
        4, "West",
        "Unknown"
    )

-- SWITCH( TRUE() ) — evaluate multiple conditions:
-- Each condition is a boolean; first TRUE wins
Performance Band =
    SWITCH( TRUE(),
        [Sales YTD] >= 1000000, "🏆 Elite",
        [Sales YTD] >= 500000, "🥇 Gold",
        [Sales YTD] >= 200000, "🥈 Silver",
        [Sales YTD] >= 50000, "🥉 Bronze",
        "🏗️ Developing"
    )

-- ✔ SWITCH(TRUE()) replaces messy nested IFs perfectly
```

% of Total	<code>DIVIDE([Measure], CALCULATE([Measure], ALL(Table)))</code>
% of Parent	<code>DIVIDE([Measure], CALCULATE([Measure], ALLEXCEPT(T,T[Col])))</code>
Running Total	<code>CALCULATE([M], FILTER(ALL('Date'),'Date'[Date]<=MAX('Date'[Date])))</code>
YoY Change %	<code>DIVIDE([M] - [M LY], [M LY], BLANK())</code>
Top N filter	<code>RANKX(ALL(T), [M], , DESC, DENSE) <= N</code>
Dynamic Title	<code>SELECTEDVALUE('Date'[Year], "All Years") & " Sales"</code>
First/Last Non Blank	<code>CALCULATE([M], LASTNONBLANK('Date'[Date],[M]))</code>
Count Distinct	<code>DISTINCTCOUNT(Sales[CustomerID])</code>
New vs Repeat	<code>IF(CALCULATE(COUNTROWS(Sales), FILTER(...)) > 1, "Repeat", "New")</code>
Conditional Format	<code>IF([M] < [Target], "#EF476F", "#06D6A0")</code>

DAX Studio (free, open-source) connects to Power BI Desktop, SSAS, and Power Pivot models to run, test, and profile DAX queries.

```
DAX QUERY SYNTAX (DAX Studio / SSMS)

-- DAX Studio uses the EVALUATE statement to run queries
EVALUATE
    SUMMARIZECOLUMNS (
        'Date'[Year],
        Products[Category],
        "Total Sales", [Total Sales],
        "Margin %",    [Profit Margin]
    )

-- Add ORDER BY:
EVALUATE
    ADDCOLUMNS (
        VALUES ( Products[Category] ),
        "Revenue", [Total Sales]
    )
ORDER BY [Revenue] DESC
```

DAX Studio Features

- ✓ Run EVALUATE queries against model
- ✓ Server timings: FE vs SE time
- ✓ Query plan analysis
- ✓ All DAX functions with IntelliSense
- ✓ Format DAX expressions
- ✓ Export results to Excel/CSV
- ✓ VertiPaq Analyzer for model size

SUMMARIZECOLUMNS

```
-- SUMMARIZECOLUMNS( groupBy1, groupBy2, ..., name, expr, ... )  
-- Preferred over SUMMARIZE in most DAX queries
```

```
-- Simple group-by with measure:
```

```
EVALUATE
```

```
    SUMMARIZECOLUMNS(  
        Products[Category],  
        "Total Sales", SUM( Sales[Amount] ),  
        "Order Count", COUNTROWS( Sales )  
    )
```

```
-- With a filter (FILTER argument):
```

```
EVALUATE
```

```
    SUMMARIZECOLUMNS(  
        Products[Category],  
        'Date'[Year],  
        FILTER( ALL('Date'), 'Date'[Year] = 2024 ),  
        "Sales", SUM( Sales[Amount] )  
    )
```

```
-- SUMMARIZE (older pattern, less efficient):
```

```
-- SUMMARIZE( table, groupCol1, groupCol2, "name", expr )
```

ADDCOLUMNS / SELECTCOLUMNS / ADDMISSINGITEMS

-- ADDCOLUMNS — adds calculated columns to an existing table

EVALUATE

```
ADDCOLUMNS (
    ALL( Products[Category] ),    -- base table
    "Revenue",    [Total Sales],
    "Margin%",    [Profit Margin],
    "Rank",       RANKX( ALL(Products), [Total Sales] )
)
```

-- SELECTCOLUMNS — pick and rename columns

EVALUATE

```
SELECTCOLUMNS (
    Products,
    "Product",    Products[ProductName],
    "Price USD",  Products[Price],
    "Revenue",    [Total Sales]
)
```

-- ADDMISSINGITEMS — forces all items to appear even with BLANK measures

EVALUATE

```
ADDMISSINGITEMS (
    'Date'[Month],
    SUMMARIZECOLUMNS ( 'Date'[Month], "Sales", [Total Sales] ),
    'Date'[Month]
)
```

CONCATENATEX / UNICHAR / OUTPUT FORMATTING

```
-- CONCATENATEX - string-join rows of a table
-- Syntax: CONCATENATEX( table, expression, delimiter [, orderBy] )
Top Regions =
    CONCATENATEX(
        TOPN( 3, ALL(Sales[Region]), [Total Sales], DESC ),
        Sales[Region],
        ", "
    )
-- Example result: "East, West, North"

-- UNICHAR - insert Unicode characters
Up Arrow    = UNICHAR(9650)    -- ▲
Down Arrow  = UNICHAR(9660)    -- ▼
Circle      = UNICHAR(9679)    -- ●

-- Trend icon measure:
Trend Icon =
    IF(
        [YoY Change] > 0,
        UNICHAR(9650) & " " & FORMAT( [YoY Change], "+0.0%;-0.0%" ),
        UNICHAR(9660) & " " & FORMAT( [YoY Change], "+0.0%;-0.0%" )
    )
```

FISCAL CALENDAR PATTERNS

```
-- Fiscal Year starting April 1 (FY April-March)
```

```
Fiscal Year =
```

```
    IF(
        MONTH( 'Date'[Date] ) >= 4,
        YEAR( 'Date'[Date] ),
        YEAR( 'Date'[Date] ) - 1
    )
```

```
Fiscal Quarter =
```

```
    SWITCH( TRUE(),
        MONTH('Date'[Date]) IN {4,5,6},    1,
        MONTH('Date'[Date]) IN {7,8,9},    2,
        MONTH('Date'[Date]) IN {10,11,12}, 3,
        4
    )
```

```
-- Fiscal YTD using custom date filter:
```

```
Fiscal YTD Sales =
```

```
VAR FYStart = DATE( [Fiscal Year], 4, 1 )
```

```
VAR Today   = MAX( 'Date'[Date] )
```

```
RETURN
```

```
    CALCULATE(
        SUM( Sales[Amount] ),
        DATESBETWEEN( 'Date'[Date], FYStart, Today )
    )
```

ABC ANALYSIS PATTERN

```
-- ABC Analysis: segment products by revenue contribution
-- A = top 70% of revenue, B = next 20%, C = bottom 10%

Cumulative % =
VAR CurrentRev  = [Total Sales]
VAR TotalRev    = CALCULATE( [Total Sales], ALL(Products) )
VAR HigherRev   =
    CALCULATE(
        [Total Sales],
        FILTER( ALL(Products), [Total Sales] >= CurrentRev )
    )
RETURN
    DIVIDE( HigherRev, TotalRev )

ABC Segment =
VAR CumPct = [Cumulative %]
RETURN
    SWITCH( TRUE(),
        CumPct <= 0.70, "A - Top 70%",
        CumPct <= 0.90, "B - Next 20%",
        "C - Bottom 10%"
    )
```

NEW VS RETURNING CUSTOMERS PATTERN

```
-- A customer is 'New' if their first purchase falls in current period  
-- A customer is 'Returning' if they purchased before
```

```
-- First Purchase Date per Customer:
```

```
First Purchase Date =  
    CALCULATE(  
        MIN( Sales[OrderDate] ),  
        ALLEXCEPT( Sales, Sales[CustomerID] )  
    )
```

```
-- New Customers (first order in current filter context):
```

```
New Customers =  
VAR PeriodStart = MIN( 'Date'[Date] )  
VAR PeriodEnd   = MAX( 'Date'[Date] )  
RETURN  
    CALCULATE(  
        DISTINCTCOUNT( Sales[CustomerID] ),  
        FILTER(  
            VALUES( Sales[CustomerID] ),  
            CALCULATE( MIN(Sales[OrderDate]) ) >= PeriodStart &&  
            CALCULATE( MIN(Sales[OrderDate]) ) <= PeriodEnd  
        )  
    )
```

```
Returning Customers = [Total Customers] - [New Customers]
```

BUDGET vs ACTUAL PATTERN

```
-- Model: separate Sales and Budget tables, both linked to Date table
-- Budget table: BudgetYear, BudgetMonth, Category, BudgetAmount
```

```
-- Actual Sales (from Sales table):
Actual Sales = SUM( Sales[Amount] )
```

```
-- Budget (from Budget table):
Budget = SUM( Budget[BudgetAmount] )
```

```
-- Absolute Variance:
Variance = [Actual Sales] - [Budget]
```

```
-- Variance % (positive = good, over budget):
Variance % = DIVIDE( [Variance], [Budget], BLANK() )
```

```
-- Conditional indicator:
Performance =
    SWITCH( TRUE(),
        [Variance %] >= 0.05,    "▲ Exceeds Budget",
        [Variance %] >= -0.05,   "= On Budget",
        "▼ Under Budget"
    )
```

```
-- YTD Budget Attainment:
YTD Attainment =
    DIVIDE(
        CALCULATE( [Actual Sales], DATESYTD('Date'[Date]) ),
        CALCULATE( [Budget], DATESYTD('Date'[Date]) ),
        BLANK()
    )
```

Learn More

- [SQLBI.com](#) : Best DAX articles & patterns
- [DAX Guide \(dax.guide\)](#) : full function reference
- [Power BI documentation \(docs.microsoft.com\)](#)
- "The Definitive Guide to DAX" Russo & Ferrari
- [Guy in a Cube YouTube channel](#)
- [RADACAD.com](#) : Advanced Power BI & DAX

Essential Tools

- [DAX Studio \(daxstudio.org\)](#) : free query/debug tool
- [VertiPaq Analyzer](#): model size analysis
- [Tabular Editor 2 \(free\)](#) : model development
- [Power BI Desktop](#) : latest monthly release
- [daxformatter.com](#) : format your DAX online
- [Bravo for Power BI](#) : date tables & formatting

Practice Projects

- Build a sales KPI dashboard from scratch
- Implement full time intelligence suite (YTD, YoY)
- Create a dynamic ABC product segmentation
- Build RLS for a multi-region dataset
- Implement a what-if scenario analysis model
- Recreate Excel financial model purely in DAX

YOU NOW KNOW



DAX

- ✓ Tabular model structure & star schema design
- ✓ Row context vs filter context
- ✓ Calculated columns vs measures
- ✓ Core aggregations: SUM, COUNT, AVERAGE, MIN/MAX
- ✓ CALCULATE : DAX's most powerful function
- ✓ ALL, ALLEXCEPT, ALLSELECTED, KEEPFILTERS
- ✓ Time intelligence: YTD, YoY, Rolling averages
- ✓ Iterator (X) functions and RANKX
- ✓ Relationship navigation & USERELATIONSHIP
- ✓ Variables (VAR/RETURN) and error handling
- ✓ Advanced patterns: ABC, new/returning, RLS, budgets

Practice daily · Profile with DAX Studio · Build real models